

Importazione dei dati da un file di solo testo

Abbiamo visto in precedenza come sono organizzati i dati in un file `.csv`, il formato raccomandato per **R**, e abbiamo visto che un file `.csv` è un semplice file di testo [1].

Se aprite con un editor di testo il file `importa_csv.csv` (se non l'avete già scaricato, trovate come farlo nella [pagina Dati](#)) questo è quello che vi appare:

```
id; sesso; anni; peso_kg; altezza_m
MT; M; 69; 76; 1,78
GF; F; 56; 63;
MC; F; 53; 71; 1,60
SB; M; 28; 73; 1,78
FE; F; 61; 54; 1,54
AB; M; 46; 92; 1,84
RF; F; 31; 81; 1,56
```

Se ci pensate bene, però, oltre a lettere, numeri, caratteri di interpunzione e qualche altro carattere accessorio, o, più tecnicamente, oltre a una serie di **caratteri ASCII stampabili** [2] nel testo è incluso un "a capo" (o se preferite un `↵` Invio) che segna il passaggio da una riga alla successiva, o, in altre parole, che separa un record dal successivo: ed è chiaro che in qualche modo questo carattere, uno dei **caratteri non stampabili** (detti anche **caratteri di controllo**) del codice ASCII, deve essere stato codificato nel file.

Che proprio questo sia il caso è dimostrabile aprendo il file con un editor di testo che sia in grado di mostrare, oltre ai caratteri ASCII, anche i caratteri di controllo contenuti nel file, uno dei quali è appunto il carattere di "a capo".

Per esempio se aprite il file `importa_csv.csv` con **PSPad** [3] ed attivate nel menù `Visualizza` la funzione `Modalità esadecimale` vedete questo:

	0001	0203	0405	0607	0809	0A0B	0C0D	0E0F	0123456789ABCDEF
00	6964	3B73	6573	736F	3B61	6E6E	693B	7065	i
10	736F	5F6B	673B	616C	7465	7A7A	615F	6D0D	d; sesso; anni; pe
20	0A4D	543B	4D3B	3639	3B37	363B	312C	3738	so_kg; altezza_m.
30	0D0A	4746	3B46	3B35	363B	3633	3B0D	0A4D	.MT; M; 69; 76; 1,78
40	433B	463B	3533	3B37	313B	312C	3630	0D0A	..GF; F; 56; 63; ..M
50	5342	3B4D	3B32	3B3B	3733	3B31	2C37	380D	C; F; 53; 71; 1,60..
60	0A46	453B	463B	3631	3B35	343B	312C	3534	SB; M; 28; 73; 1,78.
70	0D0A	4142	3B4D	3B34	363B	3932	3B31	2C38	.FE; F; 61; 54; 1,54
80	340D	0A52	463B	463B	3331	3B38	313B	312C	..AB; M; 46; 92; 1,8
90	3536								4..RF; F; 31; 81; 1,
									56

Sulla destra compaiono i caratteri ASCII contenuti nel file, sulla sinistra compare il valore esadecimale corrispondente. Sulla destra vedete ricorrere al termine di ogni record una sequenza di due punti consecutivi (`..`), sulla sinistra vedete che ad essi corrisponde la sequenza `0D0A` che è la rappresentazione esadecimale di CR (Carriage Return) e di LF (Line Feed) cioè rispettivamente del ritorno di carrello a inizio riga (`0D` esadecimale) e del passaggio ad una nuova riga (`0A` esadecimale) che l'informatica ha ereditato dalla macchina da scrivere

20	0A4D	543B	4D3B	3639	3B37	363B	312C	3738	.MT; M; 69; 76; 1,78
30	0D0A	4746	3B46	3B35	363B	3633	3B0D	0A4D	..GF; F; 56; 63; ..M
40	433B	463B	3533	3B37	313B	312C	3630	0D0A	C; F; 53; 71; 1,60..

Questi due caratteri ASCII l'editor di testo li interpreta come caratteri di controllo, cioè come caratteri che non devono essere rappresentati, ma che indicano una azione da compiere: passare a una nuova riga di testo inserendo un "a capo" (un `↵` Invio) [4].

R è ovviamente in grado di interpretare correttamente i caratteri di controllo, infatti se nella `Console` di **R** incollate questa riga di codice

```
mydatacsv <- read.table("C:/Rdati/importa_csv.csv", header=TRUE, sep=";", dec=",")
```

e premete `↵` `Invio` per importare i dati dal file `.csv`, quindi digitate

```
mydatacsv
```

e premete `↵` `Invio` per mostrare i dati importati, ottenete questo risultato:

```
> mydatacsv <- read.table("C:/Rdati/importa_csv.csv", header=TRUE, sep=";", dec=",")
> mydatacsv
  id sesso anni peso_kg altezza_m
1 MT     M   69     76     1.78
2 GF     F   56     63      NA
3 MC     F   53     71     1.60
4 SB     M   28     73     1.78
5 FE     F   61     54     1.54
6 AB     M   46     92     1.84
7 RF     F   31     81     1.56
```

(`NA` indica un dato mancante).

Ma con **R** è possibile un'altra cosa: si possono codificare i caratteri di controllo in chiaro, come viene fatto nel file `importa_txt.txt` che contiene gli stessi dati e che, aperto con un editor di testo, così vi appare:

```
MT;M;69;76;1,78\nGF;F;56;63;\nMC;F;53;71;1,60\nSB;M;28;73;1,78\nFE;F;61;54;1,54\nAB;M;46;92;1,84\nRF;F;31;81;1,56
```

Da notare che nel file `importa_txt.txt` rispetto al corrispondente file `.csv`:

→ i caratteri di controllo `CR` e `LF` (nascosti nel file `.csv`) sono stati sostituiti con i caratteri (in chiaro) `\n` ove la barra rovesciata `\` (o `backslash`) indica a **R** la presenza di un carattere di controllo e `n` sta per `New Line` ("passa a una nuova riga");

→ il file non contiene i nomi delle variabili, che devono essere specificati nella funzione **`read.table()`** mediante l'argomento **`col.names`**;

→ l'ultimo carattere nel file **deve** essere un `↵` `Invio` (cioè un "a capo") che in questo caso deve immediatamente seguire il `6` del dato finale `1,56`.

Il file `importa_txt.txt` potete scaricarlo e installarlo nella cartella `C:\Rdati` seguendo le istruzioni fornite nella [pagina Dati](#). Ora copiate nella `Console` di **R** questa riga di codice

```
mydatatxt <- read.table("c:/Rdati/importa_txt.txt", header=TRUE, col.names=c("id",  
"sesso", "anni", "peso_kg", "altezza_m"), sep=";", dec=",", allowEscapes=TRUE)
```

e premete `↵` `Invio` per importare i dati, quindi digitate

```
mydatatxt
```

e premete `↵` `Invio` per visualizzare i dati importati.

Da notare che l'argomento **`allowEscapes=TRUE`** fa sì che **R** interpreti la sequenza `\n` come un carattere di controllo (un `↵` `Invio`) e non come due semplici caratteri di testo. Come vedete il risultato ottenuto

```
> mydatatxt <- read.table("c:/Rdati/importa_txt.txt", header=TRUE,  
col.names=c("id", "sesso", "anni", "peso_kg", "altezza_m"), sep=";", dec=",",  
allowEscapes=TRUE)  
> mydatatxt
  id sesso anni peso_kg altezza_m
1 MT     M   69     76     1.78
2 GF     F   56     63      NA
3 MC     F   53     71     1.60
```

4	SB	M	28	73	1.78
5	FE	F	61	54	1.54
6	AB	M	46	92	1.84
7	RF	F	31	81	1.56

è identico a quello ottenuto importando i dati del file `.csv`.

Il senso di tutto ciò? Semplice. Guardate [il set di dati Galton](#), si tratta di 928 righe di dati, ciascuna corrispondente a un record, più una riga per i nomi delle variabili. Se invece di un file `.csv` impiegate un file `.txt` separando i record con `\n` potete compattare i dati in un numero enormemente inferiore di righe, cosa che ad esempio rende possibile inserire un numero così elevato di record in un testo.

Addendum

Se siete arrivati fino a qui, avete certamente notato che nel post manca una cosa fondamentale: un metodo semplice per generare un file di testo nel quale i caratteri di controllo nascosti `CR` e `LF` sono sostituiti con un `\n` riportato in chiaro.

In effetti ci sono programmi che consentono di effettuare la sostituzione online: alcuni forniscono come risultato un file nel quale la sequenza di caratteri di controllo `CRLF` è automaticamente sostituita con `\n` [5], altri forniscono come risultato un file nel quale detta sequenza è sostituita con uno spazio vuoto [6], che successivamente può essere sostituito manualmente con `\n`.

Qui vediamo come effettuare la sostituzione con la seconda modalità, che è la più semplice, impiegando **R** e un comune editor di testo.

Scaricate dal **CRAN** e installate il pacchetto **stringr** [7]. Quindi copiate e incollate nella Console di **R** questo script e premete ↵ Invio

```
# SALVA I DATI IN UN FILE DI TESTO ELIMINANDO I CARATTERI DI CONTROLLO CR e LF
#
library(stringr) # carica il pacchetto
mydata <- readLines("C:/Rdati/importa_csv.csv") # legge il file
newdata <- mydata[-c(1)] # elimina la prima riga che contiene i nomi delle variabili
mystring <- str_c(newdata, collapse=" ") # concatena i record in un'unica stringa separandoli
con uno spazio
write(mystring, file="C:/Rdati/temp.txt") # salva la stringa in un file di testo
```

Va subito precisato che il messaggio che vi comparirà

```
Warning message:
In readLines("C:/Rdati/importa_csv.csv") :
incomplete final line found on 'C:/Rdati/importa_csv.csv'
```

non indica un errore, ma è un semplice avvertimento che segnala la mancanza della sequenza `CRLF` (cioè di un ↵ Invio) alla fine del file: i dati sono importati correttamente, come potete constatare digitando **mydata**.

Nelle cinque righe di codice, nell'ordine:

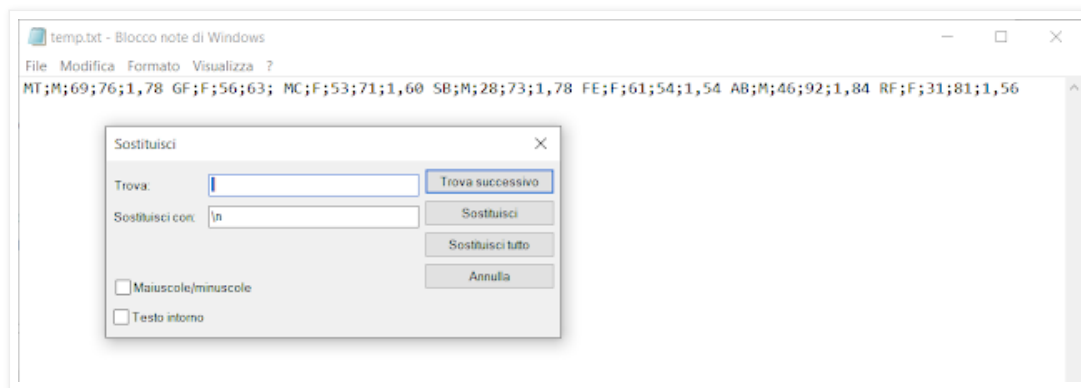
- con la funzione **library()** viene caricato il pacchetto **stringr**;
- con **readLine(s)** un normale file `.csv` (vedere il post [Importazione dei dati da un file .csv](#)) viene importato riga per riga;
- con **[-c(1)]** viene eliminata la prima riga, quella che contiene i nomi delle variabili;
- con la funzione **str_c()** del pacchetto **stringr** le righe (record) del file importate sono concatenate in un'unica stringa separando l'una dall'altra con uno spazio (**collapse=" "**) ;
- la stringa che risulta dal concatenamento viene salvata nel file `C:/Rdati/temp.txt`.

Digitate **mydata**, poi **newdata** e infine **mystring** per visualizzare la progressiva trasformazione dei dati.

Infine, impiegando un editor di testo come il Blocco note di Windows:

→ aprite il file `C:/Rdati/temp.txt`

→ utilizzate la funzione `Modifica > Sostituisci` per sostituire lo spazio vuoto con i caratteri `\n`



A questo punto ricordatevi che l'ultimo carattere del file deve essere un `↵` `Invio` (cioè un "a capo") che dovete inserire manualmente immediatamente dopo il `6` del dato finale `1,56`. Poi salvate il risultato nel file definitivo, denominato a piacere ma possibilmente per coerenza con estensione `.txt`, e avrete finalmente un file di dati di **R** in formato testo come questo

```
MT;M;69;76;1,78\nGF;F;56;63;\nMC;F;53;71;1,60\nSB;M;28;73;1,78\nFE;F;61;54;1,54\nAB;M;46;92;1,84\nRF;F;31;81;1,56
```

che impiega come separatore di record la sequenza di caratteri `\n` codificata in chiaro, e che può essere importato in **R** mediante la funzione `read.table()` impiegando l'argomento `allowEscapes=TRUE`.

Supponendo che abbiate denominato questo file `C:\Rdati\ilmiofile_txt.txt` se copiate nella Console di **R** questa riga di codice

```
mydatatxt <- read.table("c:/Rdati/ilmiofile_txt.txt", header=TRUE, col.names=c("id",  
"sesso", "anni", "peso_kg", "altezza_m"), sep=";", dec=".", allowEscapes=TRUE)
```

e premete `↵` `Invio` per importare i dati, quindi digitate

```
mydatatxt
```

e premete `↵` `Invio` per visualizzare i dati importati, potete verificare che il tutto ha funzionato correttamente e che pertanto siete in grado, al bisogno, da un lato di generare un file di testo con i caratteri di controllo in chiaro, e dall'altro lato di importarne i dati in **R**.

[1] Vedere il post [Importazione dei dati da un file .csv](#)

[2] Vedere il post [Codifica dei caratteri ASCII, ANSI, Unicode \(UTF\)](#)

[3] *PSPad freeware editor.*
<https://www.pspad.com/it/>

[4] In realtà esistono tre modi per codificare un "a capo" (un `↵` `Invio`), e dipendono dal sistema operativo impiegato:

- `CRLF (\r\n)`: usato da MS-DOS e Microsoft Windows, al quale si fa qui riferimento;
- `LF (\n)`: usato da sistemi Unix, Linux e Apple (GNU/Linux, Mac OS X e macOS);
- `CR (\r)`: usato da Commodore e Apple (Mac OS fino alla versione 9 inclusa).

[5] *Remove/Replace all line breaks of a text.*
<https://gillmeister-software.com/online-tools/text/remove-line-breaks.aspx>

[6] *Convert Newlines to Spaces.*
<https://www.browserling.com/tools/newlines-to-spaces>

[7] Vedere il manuale di riferimento del pacchetto *Package 'stringr'*.

